

Requirements Foundations

Part I

Three Types of Requirements – Functional Requirements



Functional
Requirements



Quality
Requirements



Constraints

Three Types of Requirements – Functional Requirements

Definition

- D** A functional requirement is a
- (1) “statement that identifies what results a product or process shall produce”.
 - (2) “requirement that specifies a function that a system or system component shall perform”.

[ISO/IEC/IEEE 24765]

Three Types of Requirements – Functional Requirements

Examples



Functional Requirements



FR-2: The electronic door locking system shall generate monthly reports containing all granted and denied admittances to the building.



FR-3: If the user enters a correct PIN (personal identification number) at the keypad, the system shall open the door and record the granted access (record the date and time of the access, and the name of the PIN owner).



FR-6: If a sensor detects a broken or damaged glass pane, the system shall inform the security company.

R-x: unique identifier

Three Types of Requirements – Quality Requirements

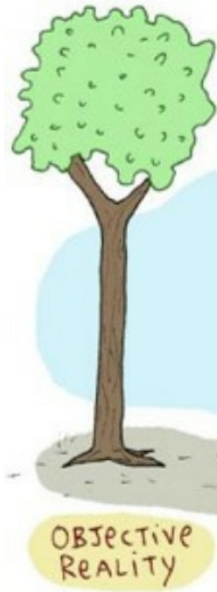


Three Types of Requirements – Quality Requirements

Definition

D A quality requirement defines a specific quality property of the entire system, of a system component, a service, or a system function.

What quality attributes does this system have?



What quality attributes does this system have?



Three Types of Requirements – Quality Requirements

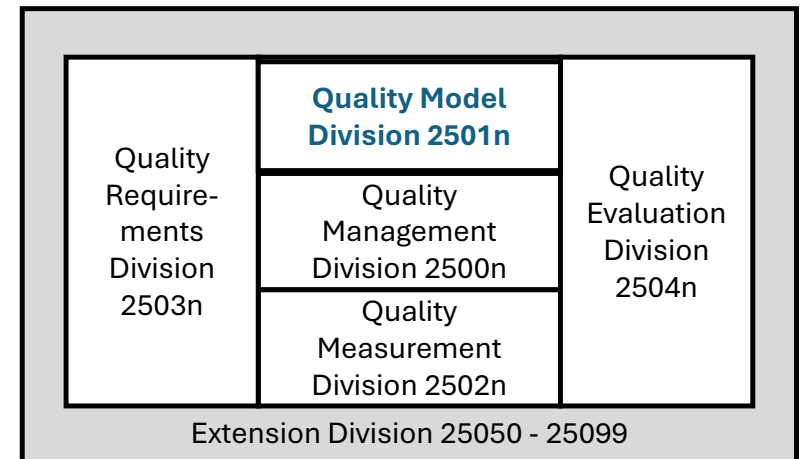
Common Classifications of Quality Requirements

- External and Internal quality:
- External quality properties can be observed and experienced by the end user(s); External quality properties include:
 - Availability, installability, integrity, interoperability, performance, reliability, robustness, safety, security, usability
- Internal quality properties are not directly observable by the end user(s); Internal quality properties include:
 - Efficiency, modifiability, portability, reusability, scalability, verifiability

Three Types of Requirements – Quality Requirements

System and Software Quality Requirements and Evaluation (SQuaRE) Framework

- SQuaRE = set of standards for assessing the quality of systems and software products
- Introduces six divisions, each consisting of several standards:
 - Quality Requirements Division (ISO/IEC 2503n)
 - Quality Model Division (ISO/IEC 2501n)
 - Quality Management (ISO/IEC 2500n)
 - Quality Measurement Division (ISO/IEC 2502n)
 - Quality Evaluation Division (ISO/IEC 2504n)
 - Extension Division (ISO/IEC 25050 – 25099)



Focus: Quality Model Division

• Based on [ISO/IEC 25030, 2019]

Three Types of Requirements – Quality Requirements

Quality Model Division (ISO/IEC 2501n)

- Quality model division consists of four ISO/IEC standards, each defining a specific quality model:
- ISO/IEC 25010 (2023) defines the product quality model
- ISO/IEC TS 25011 (2017) defines the IT service quality model
- ISO/IEC 25012 (2008) defines the data quality model
- ISO/IEC 25019 (2023) defines the quality-in-use model
- Product quality model and Quality-in-use model:
(shown on next slides)

• Quality Model Division according to ISO/IEC 25002 (2024)

Three Types of Requirements – Quality Requirements

Product Quality Model (1)

Quality Property	Description
Functional suitability	Capability of a product to provide functions that meet stated and implied needs of intended users when it is used under specified conditions
Performance efficiency	Capability of a product to perform its functions within specified time and throughput parameters and be efficient in the use of resources under specified conditions
Compatibility	Capability of a product to exchange information with other products, and/or to perform its required functions while sharing the same common environment and resources
Interaction capability	Capability of a product to be interacted with by specified users to exchange information between a user and a system via the user interface to complete the intended task
Reliability	Capability of a product to perform specified functions under specified conditions for a specified period of time without interruptions and failures

Based on [ISO/IEC 25010]

Three Types of Requirements – Quality Requirements

Product Quality Model (2)

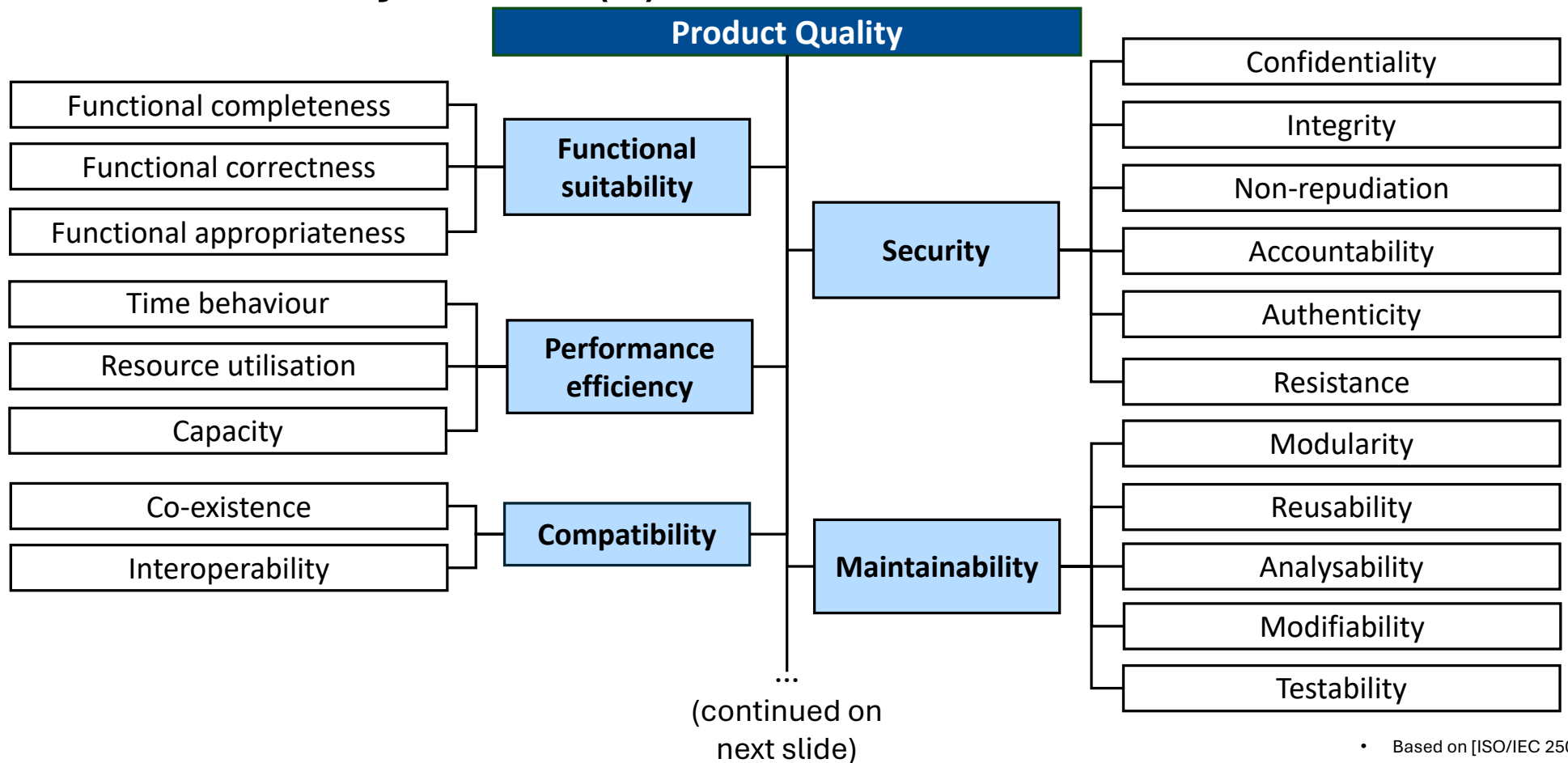
Quality Property	Description
Security	Capability of a product to protect information and data so that persons or other products have the degree of data access appropriate to their types and levels of authorization, and to defend against attack patterns by malicious actors
Maintainability	Capability of a product to be modified by the intended maintainers with effectiveness and efficiency
Flexibility	Capability of a product to be adapted to changes in its requirements, contexts of use, or system environment
Safety	Capability of a product under defined conditions to avoid a state in which human life, health, property, or the environment is endangered

<https://iso25000.com/index.php/en/iso-25000-standards/iso-25010>

Based on [ISO/IEC 25010]

Three Types of Requirements – Quality Requirements

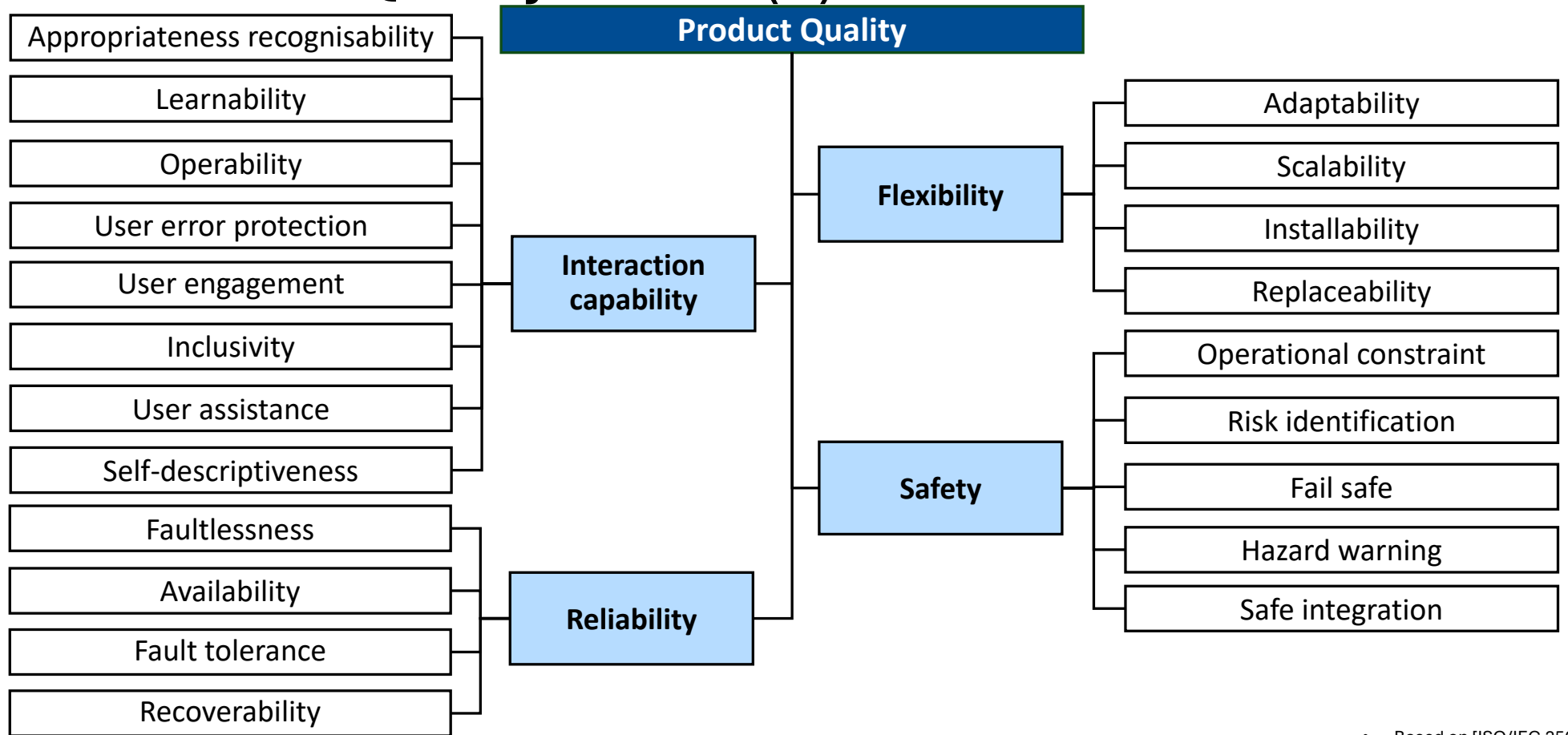
Product Quality Model (3)



• Based on [ISO/IEC 25010]

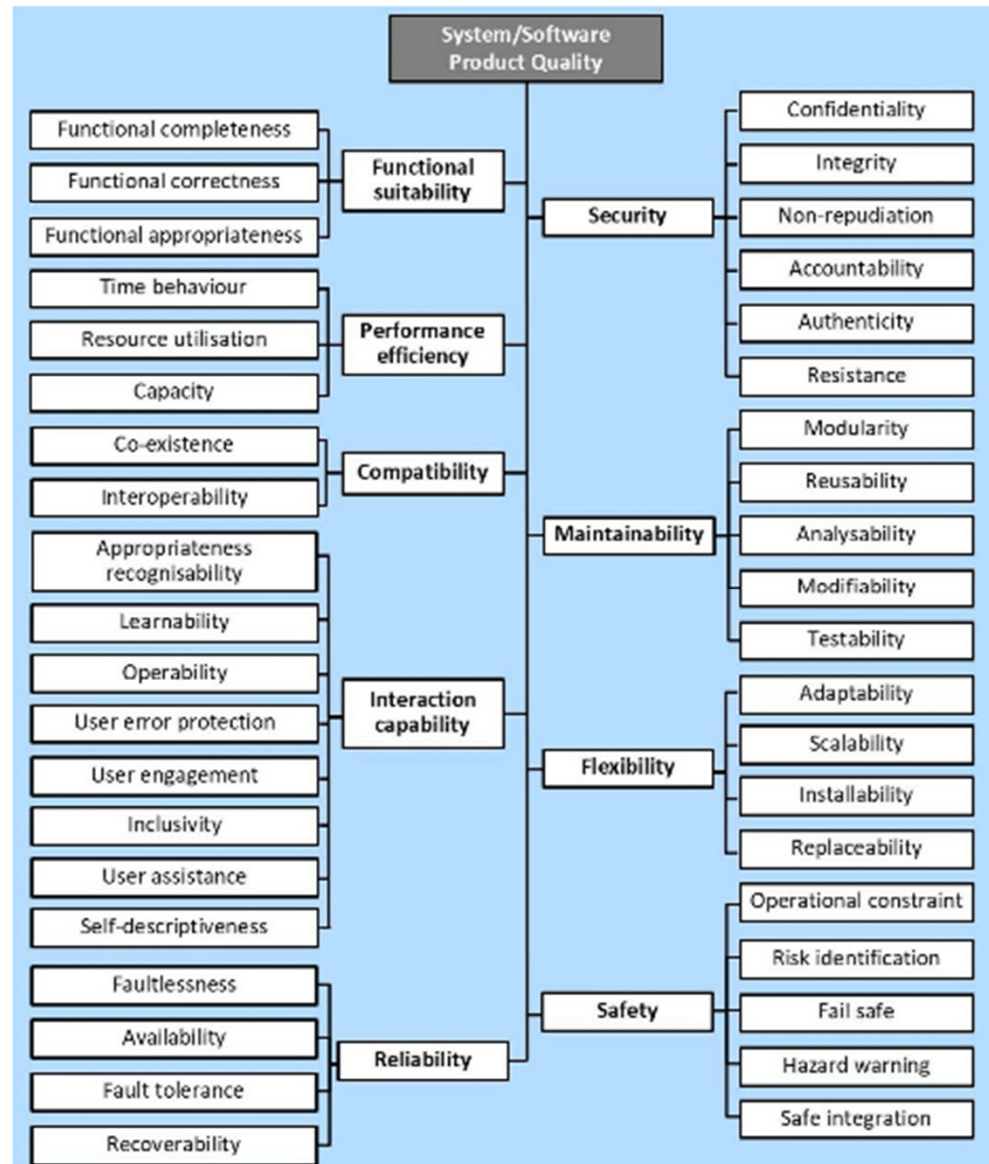
Three Types of Requirements – Quality Requirements

Product Quality Model (4)



• Based on [ISO/IEC 25010]

- Product Quality model



Three Types of Requirements – Quality Requirements

Quality-in-Use Model (1)

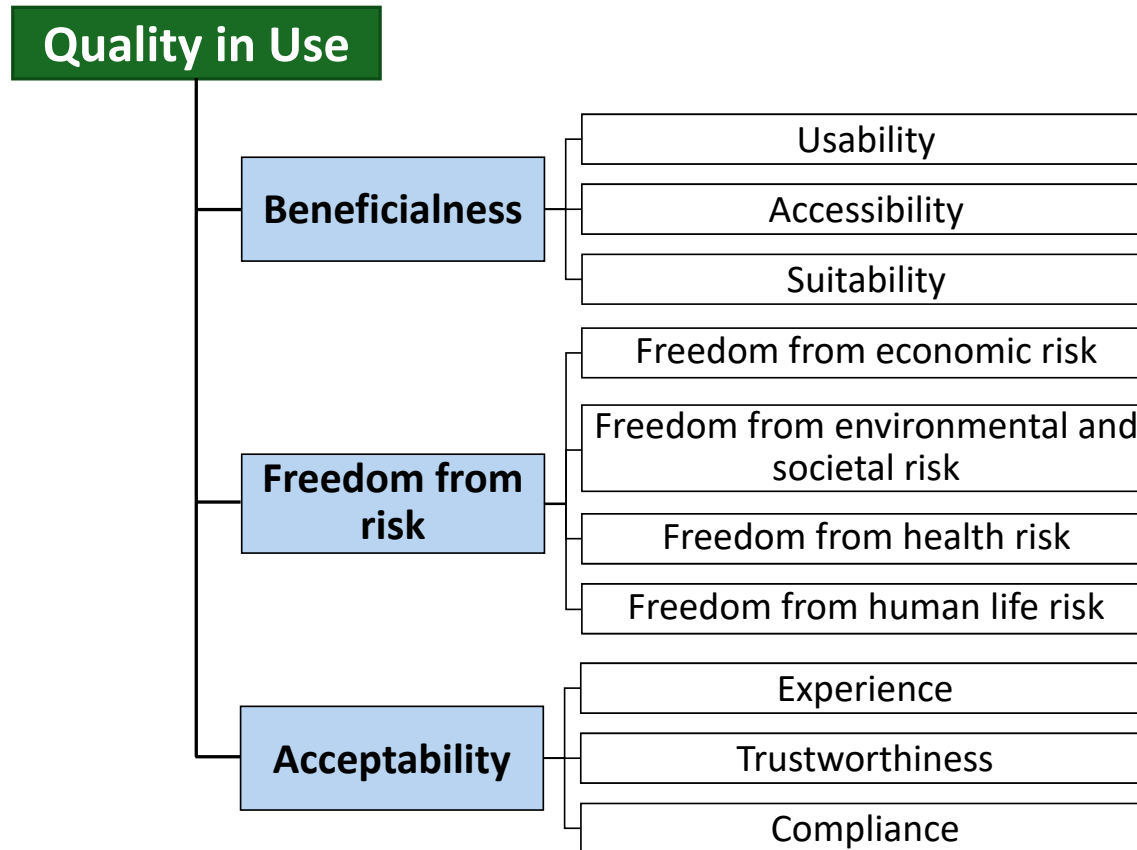
Quality Property	Description
Beneficialness	Extent of benefit resulting from the use of a product, system or service
Freedom from risk	Extent to which a product or system mitigates the potential risk to economic status, human life, health, society, financial values, enterprise activities, or the environment.
Acceptability	Extent to which a human response is favourable when accepting or installing a product, system or service software tool designed to perform some frequently used function

The quality-in-use model focuses on the stakeholder and system interactions with a software system.

Based on [ISO/IEC 25019]

Three Types of Requirements – Quality Requirements

Quality-in-Use Model (2)



- Based on [ISO/IEC 25019]

Three Types of Requirements – Quality Requirements

Examples



QR-12: The system shall keep delays below 10 percent.

Type of Quality

Freedom from risk
(Quality-in-use)



QR-15: The release of the locking mechanism shall take 0.8 seconds at most.

Performance Efficiency
(Product quality)



QR-17: The user password stored in the system shall be protected against unauthorised access.

Security
(Product quality)

Three Types of Requirements – Constraints



Three Types of Requirements – Constraints

Definition

D A constraint is an organisational or technological requirement, which restricts the way the system shall be developed, and/or the functions and properties of the system itself.

Three Types of Requirements – Constraints

Examples (1)



Constraint	Type of Constraint
 C-1: Due to current conditions defined by the insurance company, only the security technician is allowed to deactivate the control function of the system.	Organisational
 C-2: A fire protection requirement demands that the terminals in the sales rooms do not exceed the size 120 cm (height) × 90 cm (width) × 20 cm (depth).	Physical
C-3: The user interface shall not contain symbols or graphics abusive for any culture.	Culture

Three Types of Requirements – Constraints

Examples (2)



Constraint	Type of Constraint
 C-4: The system shall process personal data in compliance with the EU's Data Protection Directive 95/46/EC.	Legal
 C-5: The effort for system development shall not exceed 480 person months.	Project

Impact of Constraints

Impact of Constraints ...



... on the Development Process:

- C-16: The effort for system development shall not exceed 480 person months.

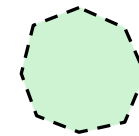
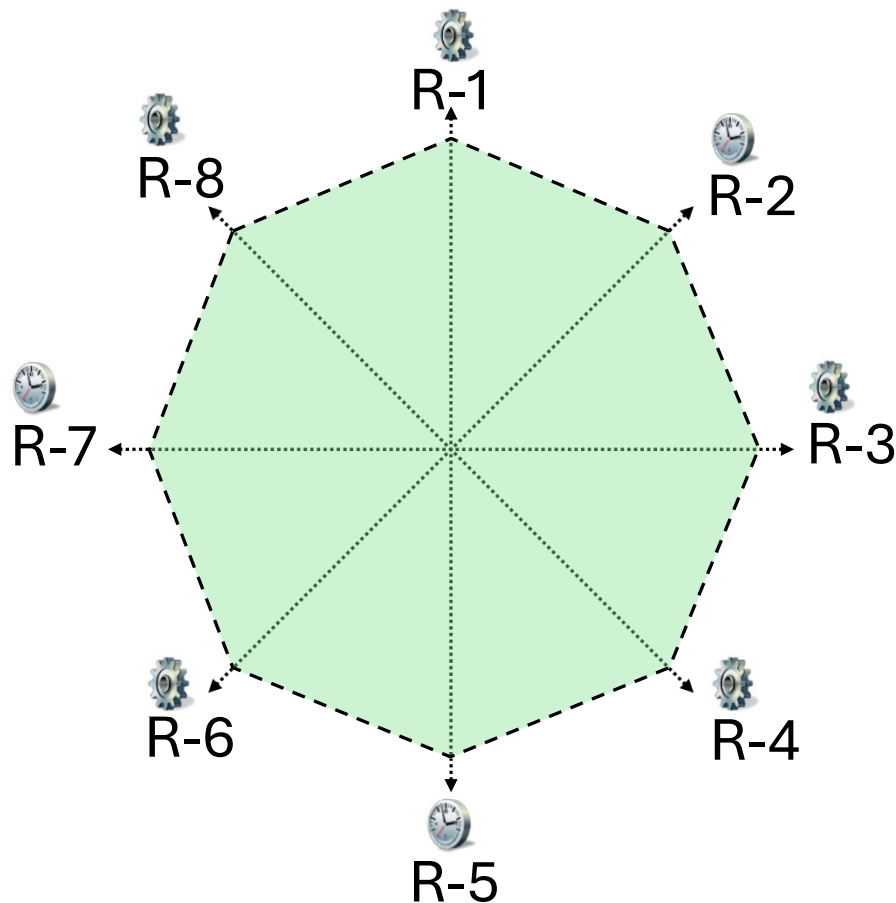
... on the System:

- C-36: The electronic control unit in the vehicle interior shall work at temperatures from -10 to +50 degrees Celsius.

Constraints typically influence both the development process and the system itself (i.e., the properties of the system).

Impact of Constraints

Restrictions imposed by Constraints (1)

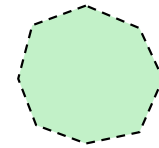
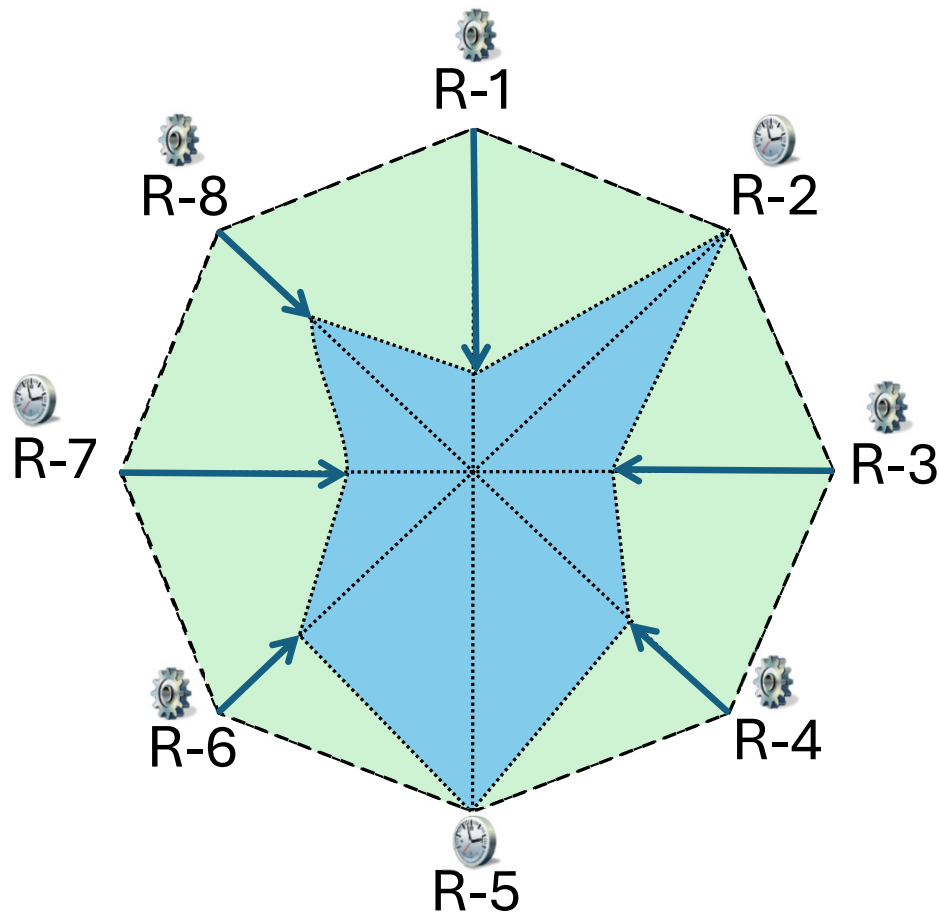


Realisation alternatives
for functional and quality
requirements **without**
constraints

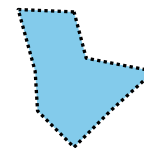


Impact of Constraints

Restrictions imposed by Constraints (2)



Realisation alternatives for functional and quality requirements **without** constraints



Reduced realisation alternatives for requirements **caused by** constraints



Reduction of valid realisation alternatives **caused by** constraints

Impact of Constraints

Example 1: Mobile App (1)

E



R-3: The output shall be presented on a mobile phone.

Possible solutions include:

iOS



Tizen



Ubuntu Touch



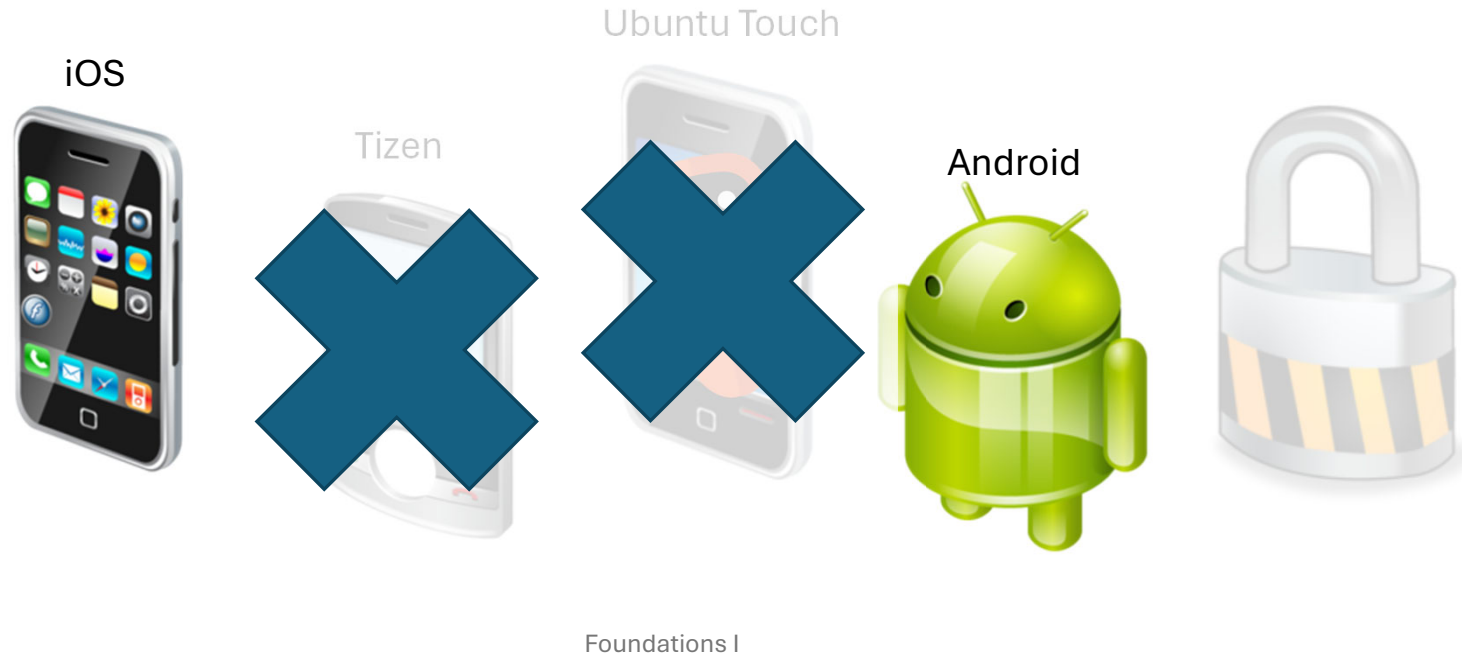
Android



Impact of Constraints

Example 1: Mobile App (2)

- E**  C-4: Only iOS and Android shall be supported.
Consequence: 2 out of 4 initial solutions left (50%)



Impact of Constraints

Example 2: Building Security System



FR-17: A personal password shall be used for authentication of each person at the access control system.

During an interview, the director of a security firm mentions that according to a governmental law access control to the building must comply with a specific standard. The standard prohibits authentication using personal passwords. Instead, it requires fingerprint authentication with a minimum accuracy of 60 minutiae (characteristic details of a fingerprint).

This constraint leads to:

Requirement change:

FR-17: A fingerprint sensor has to be used for authentication of each person at the access control system.

New requirement:

QR-4: The authentication must be performed with an accuracy of at least 60 minutiae.

So-Called “Non-Functional Requirements”

Non-functional Requirements

- The term “non-functional requirement” is **widely used** in practice and literature
- Whenever a requirement is called non-functional requirement, the requirement is typically **insufficiently understood/specified**
- Non-functional requirements often pass quality gates, because they are **categorised as non-functional**
- Non-functional requirements are either
 - Underspecified **functional requirements**,
 - Underspecified **quality requirements**, or
 - Both **underspecified functional** and **quality requirements**

So-Called “Non-Functional Requirements”

Example of a “Non-functional” Requirement

E NFR-12: The system shall be secure.

This is clearly an underspecified requirement.

It raises, among others, the following questions:

- What does “secure” mean?
- Which properties shall the system provide in order to be “secure”?
- Which functionality offered by the system should be “secure”?
- Which data the system deals with should be “secure”?
- How can one check whether the implemented system is “secure”?

So-Called “Non-Functional Requirements”

Refinement of Non-Functional Requirements

E

NFR-12: The system shall be secure.

This underspecified “non-functional” requirement should be refined into more precise functional and quality requirements, e.g.:

- R-12.1: Each user must log in to the system with his user name and password prior to using the system. (*functional requirement*)
- R-12.2: The system shall remind the user every four weeks to change the password. (*functional requirement*)
- R-12.3: When the user changes his password, the system shall validate that the new password is at least eight characters long and contains alphanumeric characters. (*functional requirement*)
- R-12.4: The user password stored in the system shall be protected against password theft. (*quality requirement - security*)

So-Called “Non-Functional Requirements”

How to Deal with Non-Functional Requirements?

- **Avoid** the **category “non-functional”** requirements in a requirements specification
- Using the term “non-functional requirements” **leads to a wrong classification** of requirements
- Only differentiate between **functional requirements** and **quality requirements**
- Refine each potential “non-functional” requirement into an adequate set of **functional requirements**, **quality requirements** or **both**

Summary

- Three types of requirements:
Functional requirements, quality requirements and constraints
- Quality of use and System/software product quality
- Impact of constraints on the system and the development of the system
- Avoid “non-functional” requirements
(and thereby avoid a key source of underspecification)

Literature References

- [Boehm and Basili 2001] B. Boehm, V. Basili: Software Defect Reduction Top 10 List. IEEE Computer, Vol. 34, No. 1, IEEE Computer Society, Los Alamitos, 2001, pp. 135-137.
- [Brooks 1987] F.P. Brooks: No Silver Bullet – Essence and Accidents of Software Engineering. IEEE Computer, Vol. 20, No. 4, IEEE Computer Society, 1987, pp. 10-19.
- [ISO/IEC 25010] ISO/IEC 25010:2011: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models, International Organization for Standardization (ISO), 2011.
- [ISO/IEC/IEEE 24765] International Organization for Standardization: Systems and Software Engineering – Vocabulary (ISO/IEC/IEEE 24765:2017), Second edition, Sept. 2017.
- [Robertson and Robertson 2013] S. Robertson, J. Robertson: Mastering the Requirements Process. 3rd edition. Addison-Wesley. Upper Saddle River, NJ. 2013,
- [The Standish Group 1995] The Standish Group Report: CHAOS. The Standish Group, 1995.
- [The Standish Group 2014] The Standish Group: CHAOS Manifesto 2014. Value versus Success & the Orthogonals.
- [The Standish Group 2015] Standish Group: CHAOS Report 2015. The Secret to Success. Retrieved April 19, 2023, from https://www.standishgroup.com/sample_research_files/CHAOSReport2015-Final.pdf accessed on 15/05/2023.

Image References

- [1] Licensed by <http://www.icons shock.com/>
- [2] Provided by Microsoft Office

Legend



Definition



Example